

Упражнение №0: Представяне на данни, Битови операции и Основи на криптографията

1. Цел на упражнението

Целта на това въвеждащо упражнение е да промени начина, по който мислите за данните – не като за текст или символи, а като за поредица от байтове и битове. В съвременната криптография, всички алгоритми работят върху двоични данни. Ще разгледаме:

- Как компютърът вижда текста (ASCII/Hex).
- Операцията **XOR** – ключова за криптографията.
- Концепцията за **One-Time Pad (OTP)**.
- Шифърът на Цезар като пример за модулна аритметика.

2. Теоретична обосновка

2.1. Представяне на данни (ASCII vs Hex)

В езика C типът `char` всъщност е малко цяло число (8-бита, 1 байт). Когато запишем символа 'A', компютърът запазва стойността **65** (десетично) или **01000001** (двоично).

За удобство в киберсигурността използваме **Шестнайсетична бройна система (Hexadecimal)**, защото тя представя един байт компактно с два символа (от **00** до **FF**).

Пример: Текстът "Cato":

- **Текст:** C | a | t | o
- **Decimal:** 67 | 97 | 116 | 111
- **Binary:** 01000011 | 01100001 | 01110100 | 01101111
- **Hex:** 0x43 | 0x61 | 0x74 | 0x6F

Важно: В C, за да принтирате число в HEX формат, използвайте форматния спецификатор `%x` или `%02x` (за подравняване).

2.2. Операцията XOR (Exclusive OR)

XOR (логическо "изключващо ИЛИ") е най-важната операция в симетричната криптография. Означава се със символа $\oplus$ (в математиката) или `^` (в езика C).

Таблица на истинност:



A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

Просто наблюдение: Еднаквите стойности дават резултат 0, а различните - 1

Криптографски свойства на XOR:

1. Обратимост: $A \oplus B \oplus B = A$.
Това означава, че ако криптираме съобщение (P) с ключ (K), получаваме шифротекст $C = P \oplus K$. За да декриптираме, прилагаме същия ключ отново: $C \oplus K = P$.
2. **Запазване на ентропията:** Ако B е напълно случаен бит (0 или 1 с вероятност 50%), то $A \oplus B$ също ще изглежда като напълно случаен бит, независимо от стойността на A .

2.3. One-Time Pad (Шифър на Вернам)

One-Time Pad (OTP) е единственият математически доказан "неразбиваем" шифър (Perfect Secrecy). Той използва XOR операция.

Алгоритъм:

1. Нека имаме съобщение P с дължина L байта.
2. Генерираме напълно случаен ключ K със същата дължина L .
3. Шифротекстът се получава чрез: $C_i = P_i \oplus K_i$ (за всеки байт i).

Защо е труден за ползване в практиката?

- Ключът трябва да е толкова дълъг, колкото съобщението.
- Ключът **никога** не трябва да се използва повторно (ако се използва, възниква атаката *Many-Time Pad*).

2.4. Шифър на Цезар (Модулна аритметика)

Шифърът на Цезар е вид **моноалфавитен шифър на заместване**, при който всяка буква в открития текст се заменя с буква, намираща се на фиксиран брой позиции (*ключ K*) по-напред в азбуката.

Математически модел

За да програмираме този шифър, първо трябва да представим буквите като числа от 0 до 25 (за английската азбука):

A = 0, B = 1, ..., Z = 25.

Нека x е номерът на буквата от открития текст, а k е ключът (отместването).

1. Криптиране (E):

$$E_k(x) = (x + k) \mod 26$$

Операцията mod 26 гарантира, че ако прехвърлим края на азбуката (например сме на 'Z' и добавим 1), ще се "завъртим" отначало (ще получим 'A').

2. Декриптиране (D):

$$D_k(x) = (x - k) \mod 26$$

Тук извършваме обратната операция – вадим ключа.

Проблеми при имплементация на C (Важно!)

1. ASCII отместване (Normalization):

Компютърът не знае, че 'A' е 0. В ASCII таблицата 'A' е 65. Ако просто приложите формулата $(\text{'A'} + 3) \% 26$, ще получите грешен резултат.

- *Стъпка 1:* Извадете кода на 'A', за да получите индекс 0-25: $p = \text{char} - \text{'A'}$;
- *Стъпка 2:* Приложете формулата: $c = (p + \text{key}) \% 26$;
- *Стъпка 3:* Върнете обратно към ASCII: $\text{result} = c + \text{'A'}$;

2. Проблемът с отрицателния модул (Decryption Bug):

При декриптиране формулата е $(x - k)$. Ако буквата е 'A' (индекс 0) и ключът е 3, получаваме -3 .

В математиката $-3 \mod 26 = 23$ (буква 'X').

В езика C обаче, операторът % върху отрицателно число връща отрицателен резултат (-3). Това ще счупи програмата ви.

- Решение: Винаги добавяйте модула преди операцията % при декриптиране:
 $\text{plain} = (\text{cipher_index} - \text{key} + 26) \% 26$;

Сигурност (Криптоанализ)

Сигурността на Цезар е особено малка. Тъй като ключът k може да бъде само число между 1 и 25 (ако е 0 или 26, текстът не се променя), съществуват само **25 възможни ключа** (при английската азбука).

- **Атака:** Brute-force (Груба сила). Можем да пробваме всички 25 ключа за части от секундата и да видим кой резултат е смислен текст.

2.5. Случайност: PRNG срещу CSPRNG

В криптографията качеството на случайните числа е критично. Ако ключът за криптиране не е наистина случаен, той може да бъде предсказан от нападател.

1. **PRNG (Pseudo-Random Number Generator):** Стандартните функции като `rand()` в C използват математически формули (напр. *Linear Congruential Generator*), които започват от начална стойност (seed).
 - **Проблемът:** Ако нападателят знае началната стойност (често това е текущото време – `time(NULL)`) или види няколко поредни генерирани числа, той може да изчисли **всички следващи** числа. Това прави `rand()` напълно неподходящ за генериране на ключове или пароли.
2. **CSPRNG (Cryptographically Secure PRNG):** Това са специални генератори (като `/dev/urandom` в Linux или `RAND_bytes` в OpenSSL), които събират "ентропия" (шум) от хардуера – движения на мишката, температурен шум на процесора, мрежови пакети. Те са непредсказуеми.

2.6. Принципът на Керкхофс

Това е златното правило на модерната криптография, формулирано от Огюст Керкхофс през 19 век.

Принцип: Сигурността на една криптосистема не трябва да зависи от запазването на алгоритъма в тайна. Сигурността трябва да зависи **единствено от пазенето на ключа в тайна**.

- **Security by Obscurity (Сигурност чрез неизвестност):** Опитът да защитим система, като крием как работи (напр. "нашият алгоритъм е фирмена тайна"). Това се счита за **лоша практика**, защото рано или късно алгоритъмът ще бъде разкрит (чрез reverse engineering или изтичане на информация) и системата ще рухне.
 - **Отворен код:** Затова алгоритми като AES и RSA са публични и проверени от хиляди експерти. Ние знаем точно как работят, но не можем да разбием съобщенията без конкретния частен ключ.
-

3. Практически насоки за реализация на C

Работа с файлове и буфери

В C езика е добра практика да четем данните байт по байт или в буфер, използвайки `unsigned char`, за да избегнем проблеми със знаковия бит при стойности над 127 (напр. 0xFF).

```
C
// Read file byte by byte
FILE *fp = fopen("input.txt", "rb"); // "rb" - read binary
unsigned char buffer;
while(fread(&buffer, 1, 1, fp) == 1) {
    // buffer contains 1 byte
    // Here we apply the cryptography
}
```

Побитови операции в C

- `&` (AND) – побитово И
- `|` (OR) – побитово ИЛИ
- `^` (XOR) – побитово изключващо ИЛИ
- `~` (NOT) – побитово отрицание

Пример за XOR криптиране на един байт:

```
C
char plaintext = 'A'; // 0x41
char key = 'S';      // 0x53
char cipher = plaintext ^ key; // Резултат: 0x12
char decrypted = cipher ^ key; // Връща 'A' (0x41)
```

4. Контролни въпроси

1. Колко байта памет заема текстът "Hello"? А колко бита са това?
2. Ако приложим XOR два пъти с един и същ ключ върху дадено число, какво ще получим?
3. Защо не е добра идея да използваме `rand()` функцията в C за генериране на криптографски ключове?
4. Каква е разликата между "сигурност чрез неизвестност" (security by obscurity) и принципа на Керкхофс?